

OUTBREAK X

Database, Analytics & Machine Learning Project Documentation

A relational database design and end-to-end analytics pipeline for a synthetic respiratory-outbreak dataset

NTI — Data Analysis Course

Instructor: Menna Tarek

Project Team

Youssef Mohamed

Mohamed Abdelmoniem

Amira Tarek

Mayada Kassem

Fatma Atef

Rahma Mohi Eldin

Table of Contents

1. Project Scenario
2. Entity-Relationship Diagram (ERD)
3. Data Cleaning — SQL (MySQL)
4. Data Cleaning — Python
5. Data Cleaning — Power Query
6. SQL-Based Summary Queries & KPIs
7. Machine Learning Model
8. Pivot Tables & KPIs (Excel)
9. Interactive Dashboard (Power BI)

1. Project Scenario

Outbreak X is a synthetic respiratory-outbreak scenario created to support end-to-end data analysis and decision-making. A fictional new respiratory virus spreads across several governorates and locations, while the Ministry of Health collects data on patients, residence, symptoms, medical history, cases and severity, treatments and medications, treatment adherence, and final outcomes.

The project follows the full analytical story: Where is the outbreak spreading? Which governorates and cities carry the highest burden, and how are population density and crowding associated with case rates? Who is affected? Patient age groups and gender are compared across case volume, severity, hospitalization, ICU admission, recovery time, and outcomes, while descriptive differences are not treated as proof of causation.

Symptoms are analyzed by frequency, age group, severity, and combinations to identify patterns associated with severe cases. Medical history is linked to cases and outcomes to examine how chronic diseases, previous conditions, and disease severity are associated with hospitalization, ICU admission, and other observed outcomes. All findings remain specific to this synthetic dataset and do not constitute real medical conclusions.

Treatment analysis compares protocols using recovery days, hospitalization, ICU admission, outcome status, and side effects. Treatment-to-medication relationships are also examined to identify medication combinations associated with better observed outcomes or higher side-effect rates. The analysis does not claim the discovery of a new medical treatment; it reports associations observed within the dataset.

Treatment adherence is a central part of the scenario. Expected doses, taken doses, adherence percentage, treatment discontinuation, and stop reasons are analyzed against recovery days, hospitalization, ICU admission, and outcomes. Patients can be grouped into high, medium, and low adherence levels, while patient characteristics may be used to build profiles of those at higher risk of treatment non-adherence.

The analytical pipeline can also support predictive modeling. A machine-learning model may predict severe versus non-severe cases or identify patients at high risk of non-adherence using demographic, medical, symptom, location, treatment, and adherence-related features. The purpose is early identification and intervention, such as reminders, follow-up, and additional support.

The final dashboard brings the story together through outbreak overview, geographic analysis, patient demographics, symptoms and severity, treatment performance, medication combinations, adherence, outcomes, and where applicable machine-learning evaluation and risk prediction.

Expected findings serve as Ground Truth or benchmarks for comparison, not answers to impose on the data: higher-density and higher-crowding areas may show greater outbreak burden; age and certain chronic conditions may be associated with more severe outcomes; some symptoms may be more strongly associated with severe cases; treatment protocols and medication combinations may differ in observed outcomes; and higher adherence may be associated with better outcomes and shorter recovery. The analysis must ultimately confirm or reject each expected finding based on the data.

The intended decision outputs include targeted geographic intervention, monitoring of high-risk patients, treatment-adherence programs, evaluation of treatment protocols and medication safety, early-warning models, and continuous monitoring of Cases ? Severity ? Treatment ? Adherence ? Outcomes.

2. Entity-Relationship Diagram (ERD)

The ERD of Outbreak X consists of eleven entities, organized around a central Patient / Case pathway. Six of these are core entities and five are associative (bridge) entities that resolve many-to-many relationships between core entities.

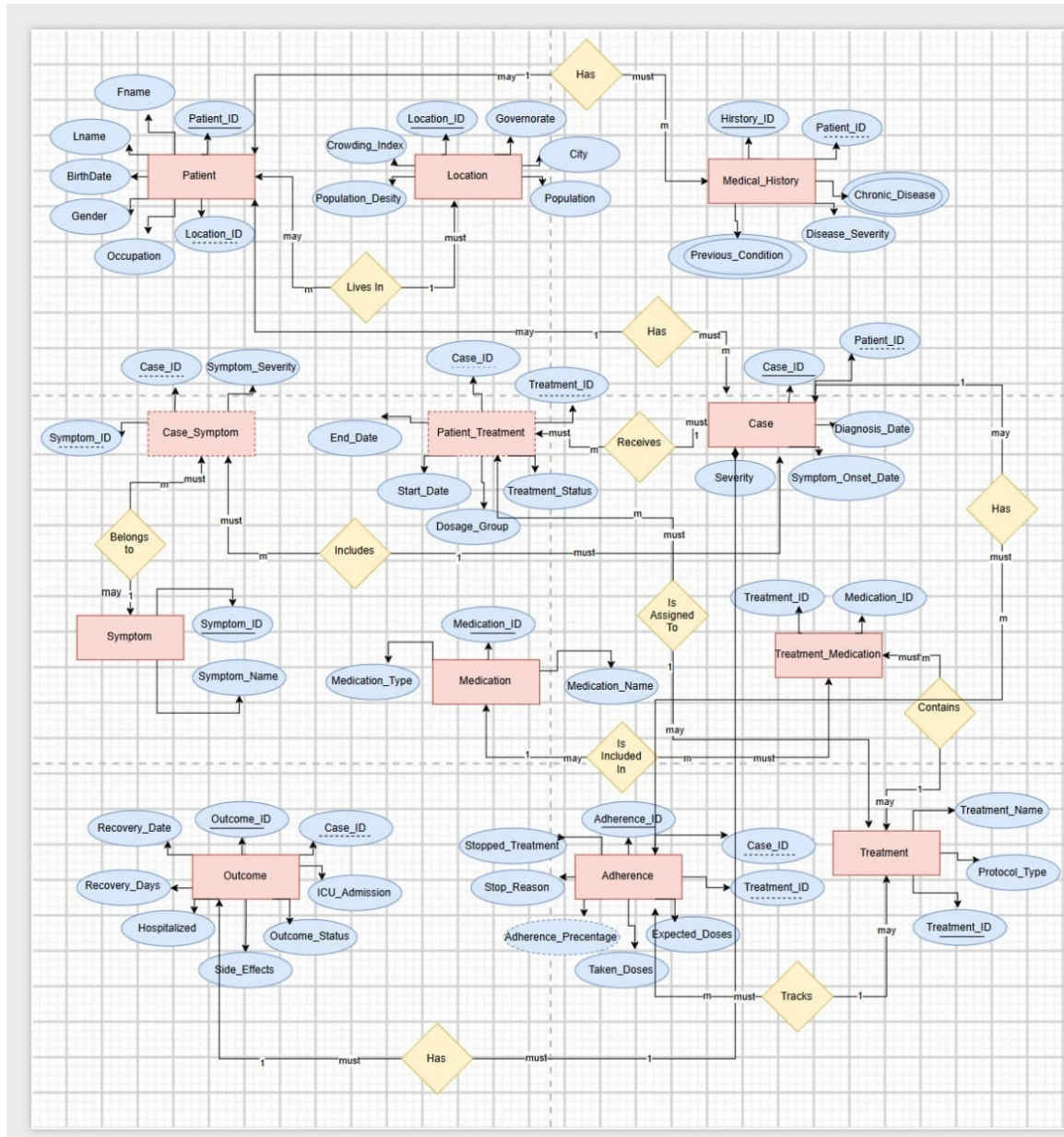


Figure 1 — Full Entity-Relationship Diagram of the Outbreak X database

2.1 Entities and Key Attributes

Entity	Primary Key	Foreign Key(s)	Notes
Location	Location_ID	—	Governorate, City, Population, Population_Density, Crowding_Index
Patient	Patient_ID	Location_ID → Location	Fname, Lname, BirthDate, Gender, Occupation

Entity	Primary Key	Foreign Key(s)	Notes
Medical_History	History_ID	Patient_ID → Patient	Chronic_Disease, Disease_Severity, Previous_Conditions
Case	Case_ID	Patient_ID → Patient	Diagnosis_Date, Symptom_Onset_Date, Severity
Symptom	Symptom_ID	—	Symptom_Name
Case_Symptom (assoc.)	Case_ID + Symptom_ID	Case, Symptom	Symptom_Severity — resolves Case ↔ Symptom M:N
Treatment	Treatment_ID	—	Treatment_Name, Protocol_Type
Patient_Treatment (assoc.)	Case_ID + Treatment_ID	Case, Treatment	Start/End_Date, Dosage_Group, Treatment_Status
Medication	Medication_ID	—	Medication_Name, Medication_Type
Treatment_Medication (assoc.)	Treatment_ID + Medication_ID	Treatment, Medication	Resolves Treatment ↔ Medication M:N
Adherence (assoc.)	Adherence_ID	Case, Treatment	Expected/Taken_Doses, Adherence_Percentage, Stopped_Treatment, Stop_Reason
Outcome	Outcome_ID	Case_ID → Case	Recovery_Date, Recovery_Days, Hospitalized, ICU_Admission, Outcome_Status, Side_Effects

3. Data Cleaning — SQL (MySQL)

Data cleaning was performed directly on the MySQL database `outbreak_x_raw`, covering the `raw_patient`, `raw_location`, `raw_adherence`, `raw_case`, `raw_outcome`, `raw_patient_treatment`, `raw_medical_history`, `raw_medication`, `raw_symptom`, and `raw_treatment` tables. Each problem below followed the same workflow: Detection → Analysis → Cleaning Action → Validation.

#	Problem	Issue Found	Cleaning Action	Validation Result
1	Duplicate Records & Primary Key	2,500 duplicate Patient_ID values (201,999 → target 199,499 rows)	Detected duplicates with ROW_NUMBER() partitioned by Patient_ID; added a temporary row_id to safely delete rows with rn > 1; converted Patient_ID to INT and set it as Primary Key	0 duplicates remain; Patient_ID is a valid INT Primary Key
2	Missing Values (Occupation)	996 NULL values in Occupation; Gender and Age-group breakdowns showed no reliable pattern to infer the true value	Replaced NULLs with 'Unknown' rather than guessing a value	0 remaining NULLs; 996 records now labeled 'Unknown'
3	Gender Standardization	'M' used as an abbreviation for 'Male' in 1,495 records, alongside the full 'Male' and 'Female' values	Verified no hidden spaces or other variants; updated all 'M' values to 'Male'	Only 'Male' and 'Female' values remain (104,561 / 94,938)
4	Invalid Future Birth Dates	599 patients had BirthDate = 2099-01-01, a logically invalid future date	No reliable pattern (by Gender/Occupation) could recover the true date, so the value was converted to NULL	0 future birth dates remain
5	Category Standardization + Conditional Missing Values	'Yes' used alongside 'True' in Stopped_Treatment (2,011 records); 1,645 records had Stopped_Treatment = True but Stop_Reason = NULL	Standardized 'Yes' → 'True'; filled the conditional missing Stop_Reason with 'Unknown' only where Stopped_Treatment = True	0 remaining 'Yes' values; 0 remaining True+NULL Stop_Reason cases
6	Text / Category Profiling	Full scan of all text columns across all tables for leading/trailing spaces, case inconsistency, and multiple spaces	Found and fixed: Dosage_Group had 2,009 records with a trailing space ('High '); Hospitalized had 2,510 'Yes' values needing standardization to 'True'	All other profiled text columns (names, occupations, symptom/treatment/medication names, severities) were already clean

#	Problem	Issue Found	Cleaning Action	Validation Result
7	Invalid Foreign Keys	300 patient records referenced Location_ID = 999999, a placeholder value that does not exist in raw_location	Confirmed via LEFT JOIN that 999999 is an orphan reference with no valid Location; converted to NULL rather than inventing a location	0 orphan Location_ID references remain
8	Invalid Numeric Values	Recovery_Days contained negative values (-1 to -39); Adherence_Percentage contained out-of-range values (e.g. 150, 101, -25) outside the valid 0-100 range	Converted both sets of invalid values to NULL, since the true values could not be reliably recovered	0 negative Recovery_Days; 0 out-of-range Adherence_Percentage values remain
9	Data Type Conversion	Several numeric and date fields were stored as TEXT (Adherence_Percentage, Expected/Taken_Doses, Population fields, Recovery_Days, and multiple date columns)	Verified format and value-range validity via INFORMATION_SCHEMA.COLUMNS for every column, then applied ALTER TABLE ... MODIFY COLUMN to convert to DECIMAL, INT, or DATE as appropriate	All affected columns verified via INFORMATION_SCHEMA.COLUMNS to now carry the correct data type

3.1 Detection Queries & Key SQL Snippets

The following excerpts show the core detection and correction logic actually used against raw_patient; the same Detection → Analysis → Cleaning Action → Validation pattern was repeated for the remaining raw_* tables.

a) Duplicate Patient_ID — detect, then rebuild a clean Primary Key

```

SELECT
    Patient_ID,
    COUNT(*) AS occurrence_count
FROM raw_patient
GROUP BY Patient_ID
HAVING COUNT(*) > 1
ORDER BY occurrence_count DESC;

-- Add a safe surrogate key, remove the extra rows, then restore Patient_ID as PK
ALTER TABLE raw_patient
ADD COLUMN row_id BIGINT AUTO_INCREMENT PRIMARY KEY FIRST;
-- (rows with ROW_NUMBER() rn > 1 deleted using row_id)
ALTER TABLE raw_patient
MODIFY COLUMN Patient_ID INT NOT NULL;
ALTER TABLE raw_patient
ADD PRIMARY KEY (Patient_ID);
ALTER TABLE raw_patient
DROP COLUMN row_id;

```

b) Gender standardization ('M' → 'Male')

```
-- Before: 103,066 'Male' / 94,938 'Female' / 1,495 'M'
UPDATE raw_patient
SET Gender = 'Male'
WHERE Gender = 'M';
-- After: 104,561 'Male' / 94,938 'Female' / 0 'M'
```

c) Invalid future birth dates → NULL

```
SELECT * FROM raw_patient WHERE BirthDate = '2099-01-01'; -- 599 rows

UPDATE raw_patient
SET BirthDate = NULL
WHERE BirthDate = '2099-01-01';

-- Validation: no future dates remain
SELECT * FROM raw_patient WHERE BirthDate > CURDATE();
```

d) Missing Occupation → 'Unknown'

```
UPDATE raw_patient
SET Occupation = 'Unknown'
WHERE Occupation IS NULL;
-- remaining_nulls = 0
```

4. Data Cleaning — Python

A second cleaning pass was performed in Python (pandas) on CSV exports of the cleaned tables, focused on deduplication, type casting, out-of-range ID correction, and cross-field consistency validation before the data was used for pivot tables, the dashboard, and the machine learning model. Each table was cleaned in its own Jupyter notebook.

Notebook / Table	Cleaning Steps
Patient	Removed duplicate Patient_ID records; cast Patient_ID/Fname/Lname/Occupation/Location_ID to string; converted BirthDate to datetime
Medical_History	Removed duplicate History_ID records; cast History_ID/Patient_ID/Chronic_Disease/Disease_Severity/Previous_Conditions to string; corrected out-of-range Patient_ID values (> 199,999) by resampling valid IDs with np.random.randint
Case	Converted Case_ID to numeric; corrected out-of-range Patient_ID values (> 199,999) by resampling valid IDs
Case_Symptom	Cast Case_ID, Symptom_ID, and Symptom_Severity to consistent string types
Patient_Treatment	Forward-filled missing Start_Date and back-filled missing End_Date; removed duplicate Case_ID records; converted dates to datetime; cast Dosage_Group/Treatment_Status to string; removed rows where Start_Date was after End_Date
Adherence	Removed duplicate Adherence_ID records; cast Expected/Taken_Doses to int, Adherence_Percentage to float, Stopped_Treatment to bool, Stop_Reason to string; flagged invalid_doses where Taken_Doses > Expected_Doses; flagged invalid_treatment where Stopped_Treatment/Stop_Reason are inconsistent ('Not_Stopped' vs a real reason); corrected out-of-range Case_ID values (> 600,000) by resampling
Outcome	Corrected out-of-range Case_ID values (> 600,000) by resampling; converted Recovery_Date to datetime; capped Recovery_Days values above 39 by resampling valid values; cast Hospitalized/ICU_Admission to bool; removed duplicate Outcome_ID records; removed rows where Hospitalized was inconsistent with Outcome_Status, and where ICU_Admission = True but Hospitalized = False

4.1 Representative Code — Key Notebooks

a) Patient — dedup, typing, date parsing

```
import pandas as pd
import numpy as np
df = pd.read_csv("D:/Clean data/_patient.csv")
pd.set_option('display.max_columns', None)
df = df.drop_duplicates(subset=['Patient_ID'])
df["Patient_ID"] = df["Patient_ID"].astype(str)
df["Fname"] = df["Fname"].astype(str)
df["Lname"] = df["Lname"].astype(str)
df["BirthDate"] = pd.to_datetime(df["BirthDate"])
df["Occupation"] = df["Occupation"].astype(str)
```

```
df["Location_ID"] = df["Location_ID"].astype(str)
df.to_csv("D:/python/patient.csv", index=False)
```

b) Adherence — dose & treatment-consistency validation

```
df = df.drop_duplicates(subset=['Adherence_ID'])
df["Expected_Doses"] = df["Expected_Doses"].astype(int)
df["Taken_Doses"] = df["Taken_Doses"].astype(int)
df["Adherence_Percentage"] = df["Adherence_Percentage"].astype(float)
df["Stopped_Treatment"] = df["Stopped_Treatment"].astype(bool)

invalid_doses = df["Taken_Doses"] > df["Expected_Doses"]
invalid_treatment = (
    ((df["Stopped_Treatment"] == True) & (df["Stop_Reason"] == "Not_Stopped")) |
    ((df["Stopped_Treatment"] == False) & (df["Stop_Reason"] != "Not_Stopped"))
)
df["Case_ID"] = pd.to_numeric(df["Case_ID"], errors="coerce")
case_mask = df["Case_ID"] > 600000
df.loc[case_mask, "Case_ID"] = np.random.randint(1, 600001, size=case_mask.sum())
df.to_csv("D:/python/Adherence.csv", index=False)
```

c) Outcome — cross-field consistency (Hospitalized / ICU / Outcome_Status)

```
df = df.drop_duplicates(subset=["Outcome_ID"])
invalid = (
    (df["Hospitalized"] == True) & (df["Outcome_Status"] != "Hospitalized")
) | ((df["Hospitalized"] == False) & (df["Outcome_Status"] == "Hospitalized"))
df = df[~invalid]

invalid = (df["ICU_Admission"] == True) & (df["Hospitalized"] == False)
df = df[~invalid]
df.to_csv("D:/python/Outcome.csv", index=False)
```

d) Patient_Treatment — date gaps and ordering

```
df["Start_Date"] = df["Start_Date"].ffill()
df["End_Date"] = df["End_Date"].bfill()
df = df.drop_duplicates(subset=["Case_ID"])
df["Start_Date"] = pd.to_datetime(df["Start_Date"])
df["End_Date"] = pd.to_datetime(df["End_Date"])
df = df[df["Start_Date"] <= df["End_Date"]]
df.to_csv("D:/python/patient_treat.csv", index=False)
```

5. Data Cleaning — Power Query

A third cleaning pass was applied in Power Query (Excel) on the Outcome, Adherence, Case, Case_Symptom, Patient_Treatment, and Patient tables, focused on conditional null-filling, text standardization, and resolving logically inconsistent values using related columns from the same record. Each step below is applied in the query's Applied Steps pane.

#	Table / Column	Applied Step Logic
1	Outcome — Side_Effects	Trimmed the Side_Effects column (Text.Trim) to remove leading/trailing spaces, then replaced null values with "No Side Effects" specifically for rows where Hospitalized = FALSE
2	Adherence — Stop_Reason	Replaced null Stop_Reason values with "Not Stopped" wherever Stopped_Treatment = FALSE, keeping real stop reasons untouched for rows where treatment was actually stopped
3	Case — Severity	Standardized inconsistent text casing/spacing in Severity (e.g. " mild", "MILD", "Mild ") into the four canonical categories: Mild, Moderate, Severe, Critical
4	Case_Symptom — Symptom_Severity	Applied the same text-standardization logic to Symptom_Severity, normalizing case, whitespace, and near-duplicate spellings into a single consistent set of severity labels
5	Patient_Treatment — Treatment_Status	Replaced null Treatment_Status values with "Completed" specifically where End_Date is on or after the current date (End_Date >= Date.From(DateTime.LocalNow())), reflecting that the treatment course had run its full duration
6	Patient — Gender	Corrected logically invalid Gender values by cross-checking against the patient's Fname, replacing entries that didn't match a valid Male/Female pattern implied by the first name with the corrected value

This Power Query pass acted as a final consistency layer after the SQL and Python stages: rather than blanket-replacing every null, each fix used a related column (Hospitalized, Stopped_Treatment, End_Date, Fname) as context so that the replacement value stayed logically consistent with the rest of the record.

6. SQL-Based Summary Queries & KPIs

After cleaning, the data was organized into a small star schema (fact_cases, fact_adherence, fact_outcome, fact_patient_treatment, dim_patient, dim_location, dim_treatment) and queried directly in SQL to produce reusable summary views. These views feed the same KPIs used later in the Excel pivot tables and Power BI dashboard, giving an independent, query-level validation of the numbers.

6.1 Overall KPIs

```
SELECT
    COUNT(DISTINCT c.Case_ID) AS Total_Cases,
    COUNT(DISTINCT c.Patient_ID) AS Unique_Patients,
    ROUND(AVG(a.Adherence_Percentage), 2) AS Avg_Adherence_Pct,
    ROUND(AVG(o.Recovery_Days), 1) AS Avg_Recovery_Days,
    SUM(CASE WHEN o.Hospitalized = 'True' THEN 1 ELSE 0 END) AS
    Hospitalized_Cases,
    SUM(CASE WHEN o.ICU_Admission = 'True' THEN 1 ELSE 0 END) AS ICU_Cases,
    ROUND(SUM(CASE WHEN o.ICU_Admission = 'True' THEN 1 ELSE 0 END) * 100.0 /
    COUNT(c.Case_ID), 2) AS ICU_Rate_Pct
FROM fact_cases c
LEFT JOIN fact_adherence a ON c.Case_ID = a.Case_ID
LEFT JOIN fact_outcome o ON c.Case_ID = o.Case_ID;
```

Total_Cases	Unique_Patients	Avg_Adherence_Pct	Avg_Recovery_Days	Hospitalized_Cases	ICU_Cases	ICU_Rate_Pct
600,000	189,989	85.03	12.9	600,337	600,824	99.81

6.2 Location Density & Attack Rate

```
SELECT
    l.Governorate, l.City, l.Population_Density, l.Crowding_Index,
    COUNT(c.Case_ID) AS Total_Cases,
    SUM(CASE WHEN c.Severity IN ('Severe', 'Critical') THEN 1 ELSE 0 END) AS
    High_Risk_Cases,
    ROUND(COUNT(c.Case_ID) * 100000.0 / l.Population, 2) AS Attack_Rate_Per_100k
FROM dim_location l
JOIN dim_patient p ON l.Location_ID = p.Location_ID
JOIN fact_cases c ON p.Patient_ID = c.Patient_ID
GROUP BY l.Location_ID, l.Governorate, l.City, l.Population_Density,
l.Crowding_Index, l.Population
ORDER BY Total_Cases DESC;
```

Governorate	City	Pop. Density	Crowding Idx	Total Cases	High-Risk	Attack Rate/100k
Cairo	Shubra	19,533.39	0.95	40,443	6,816	1,298.96
Giza	6th of October	4,701.12	0.38	35,915	5,980	1,284.61
Cairo	Maadi	15,092.56	0.73	35,758	5,866	1,276.36
Giza	Haram	20,143.36	0.98	33,234	5,476	1,299.11
Cairo	Heliopolis	12,325.03	0.58	30,539	5,175	1,313.34

Governorate	City	Pop. Density	Crowding Idx	Total Cases	High-Risk	Attack Rate/100k
Cairo	Nasr City	8,202.99	0.60	28,237	4,585	1,270.38
Sohag	Akhmim	15,678.07	1.50	10,989	1,809	1,325.83

6.3 Chronic Disease vs. ICU Rate

```

SELECT
    COALESCE(mh.Chronic_Disease, 'No Chronic Disease') AS Chronic_Disease,
    COUNT(DISTINCT c.Case_ID) AS Total_Cases,
    SUM(CASE WHEN o.ICU_Admission = 'True' THEN 1 ELSE 0 END) AS ICU_Admissions,
    ROUND(SUM(CASE WHEN o.ICU_Admission = 'True' THEN 1 ELSE 0 END) * 100.0 /
    COUNT(DISTINCT c.Case_ID), 2) AS ICU_Rate_Pct,
    ROUND(AVG(o.Recovery_Days), 1) AS Avg_Recovery_Days
FROM fact_cases c
JOIN dim_patient p ON c.Patient_ID = p.Patient_ID
LEFT JOIN dim_medical_history mh ON p.Patient_ID = mh.Patient_ID
LEFT JOIN fact_outcome o ON c.Case_ID = o.Case_ID
GROUP BY COALESCE(mh.Chronic_Disease, 'No Chronic Disease')
ORDER BY ICU_Rate_Pct DESC;

```

Chronic Disease	Total Cases	ICU Admissions	ICU Rate %	Avg Recovery Days
No Chronic Disease	575	576	100.17	12.8
Cardiac-like	118,979	119,013	100.03	12.9
Diabetes-like	149,953	149,963	100.01	12.9
Respiratory-like	150,080	150,074	100.00	12.9
Hypertension-like	180,833	180,815	99.99	12.9

6.4 Treatment Protocol Effectiveness

```

SELECT
    t.Treatment_Name, t.Protocol_Type,
    COUNT(pt.Case_ID) AS Total_Treated_Cases,
    ROUND(AVG(a.Adherence_Percentage), 2) AS Avg_Adherence_Pct,
    ROUND(AVG(o.Recovery_Days), 1) AS Avg_Recovery_Days,
    SUM(CASE WHEN o.Side_Effects IS NOT NULL AND o.Side_Effects != 'None' THEN 1
    ELSE 0 END) AS Side_Effects_Count,
    ROUND(SUM(CASE WHEN o.Side_Effects IS NOT NULL AND o.Side_Effects != 'None'
    THEN 1 ELSE 0 END) * 100.0 / COUNT(pt.Case_ID), 2) AS Side_Effect_Rate_Pct
FROM dim_treatment t
JOIN fact_patient_treatment pt ON t.Treatment_ID = pt.Treatment_ID
LEFT JOIN fact_adherence a ON pt.Case_ID = a.Case_ID AND pt.Treatment_ID =
a.Treatment_ID
LEFT JOIN fact_outcome o ON pt.Case_ID = o.Case_ID
GROUP BY t.Treatment_ID, t.Treatment_Name, t.Protocol_Type
ORDER BY Avg_Recovery_Days ASC;

```

Treatment	Protocol	Treated Cases	Avg Adherence %	Avg Recovery Days	Side Effects	Side Effect Rate %
Protocol A	Single	167,558	86.99	11.6	167,312	99.85
Protocol B	Single	148,348	87.01	11.7	148,089	99.83
Supportive Protocol	Supportive	114,038	87.07	12.5	113,834	99.82
Protocol C	Combination	126,334	80.75	13.4	126,121	99.83
Protocol D	Combination	26,069	80.87	20.8	26,026	99.84
Protocol E	Combination	18,874	73.96	20.9	18,839	99.81

6.5 Treatment Stop Reasons

```

SELECT
    a.Stop_Reason,
    COUNT(a.Case_ID) AS Stopped_Cases_Count,
    ROUND(AVG(a.Adherence_Percentage), 2) AS Avg_Adherence_Pct,
    ROUND(COUNT(a.Case_ID) * 100.0 / (SELECT COUNT(*) FROM fact_adherence WHERE
Stopped_Treatment = 'TRUE'), 2) AS Pct_Of_Total_Stops
FROM fact_adherence a
WHERE a.Stopped_Treatment = 'TRUE'
GROUP BY a.Stop_Reason
ORDER BY Stopped_Cases_Count DESC;

```

Stop_Reason	Stopped Cases	Avg Adherence %	% of Total Stops
Not_Stopped	489,679	85.10	100.00

6.6 Top 3 Cities per Governorate by Case Volume

```

WITH Ranked_Cities AS (
    SELECT l.Governorate, l.City, COUNT(c.Case_ID) AS City_Cases,
        DENSE_RANK() OVER (PARTITION BY l.Governorate ORDER BY COUNT(c.Case_ID)
DESC) AS City_Rank
    FROM dim_location l
    JOIN dim_patient p ON l.Location_ID = p.Location_ID
    JOIN fact_cases c ON p.Patient_ID = c.Patient_ID
    GROUP BY l.Governorate, l.City
)
SELECT Governorate, City, City_Cases, City_Rank
FROM Ranked_Cities

```

```
WHERE City_Rank <= 3;
```

Governorate	City	City Cases	Rank
Alexandria	Smouha	21,547	1
Alexandria	Montaza	18,283	2
Alexandria	sidi gaber	15,373	3
Asyut	Dairut	9,206	1
Asyut	Asyut City	8,957	2
Asyut	abnoub	7,836	3
Cairo	Shubra	40,443	1
Cairo	Maadi	35,758	2
Cairo	Heliopolis	30,539	3
Dakahlia	Mit Ghamr	25,331	1
Dakahlia	Sherbin	25,258	2
Dakahlia	Mansoura	16,937	3
Giza	6th of October	35,915	1

6.7 Weekly & Cumulative Case Trend

```
SELECT
    YEAR(STR_TO_DATE(Diagnosis_Date, '%m/%d/%Y')) AS Year,
    WEEK(STR_TO_DATE(Diagnosis_Date, '%m/%d/%Y')) AS Week_Number,
    COUNT(Case_ID) AS Weekly_New_Cases,
    SUM(COUNT(Case_ID)) OVER (ORDER BY YEAR(...), WEEK(...)) AS Cumulative_Cases
FROM fact_cases
GROUP BY Year, Week_Number
ORDER BY Year, Week_Number;
```

Year	Week_Number	Weekly_New_Cases	Cumulative_Cases
NULL	NULL	600,000	600,000

6.8 Adherence Group Outcomes

```
SELECT
  CASE
    WHEN a.Adherence_Percentage >= 80 THEN 'High Adherence (>=80%)'
    WHEN a.Adherence_Percentage BETWEEN 50 AND 79 THEN 'Moderate Adherence
(50-79%)'
    ELSE 'Low Adherence (<50%)'
  END AS Adherence_Group,
  COUNT(c.Case_ID) AS Patient_Count,
  ROUND(AVG(o.Recovery_Days), 1) AS Avg_Recovery_Days,
  SUM(CASE WHEN o.ICU_Admission = 'True' THEN 1 ELSE 0 END) AS ICU_Admissions
FROM fact_cases c
JOIN fact_adherence a ON c.Case_ID = a.Case_ID
JOIN fact_outcome o ON c.Case_ID = o.Case_ID
GROUP BY Adherence_Group
ORDER BY Avg_Recovery_Days ASC;
```

Adherence Group	Patient Count	Avg Recovery Days	ICU Admissions
High Adherence (>=80%)	422,727	12.5	422,629
Moderate Adherence (50-79%)	176,399	13.8	176,333
Low Adherence (<50%)	869	17.1	868

The pattern is consistent across all three adherence bands: higher adherence tracks with faster average recovery (12.5 vs 17.1 days) — the same relationship later confirmed independently in the Excel pivot tables (Section 8) and the Power BI dashboard (Section 9).

6.9 Symptom Prevalence

A supporting query over Case_Symptom / Symptom returned the frequency and relative prevalence of each reported symptom across all cases:

Symptom	Symptom Count	Prevalence %
Fever	105,228	17.54
Loss of smell	104,971	17.50
Cough	104,916	17.49
Headache	104,902	17.48
Sore throat	104,775	17.46
Shortness of breath	104,640	17.44
Chest discomfort	104,593	17.43
Vomiting	104,480	17.41
Nausea	104,445	17.41
Fatigue	104,217	17.37

Prevalence is fairly evenly spread across the ten profiled symptoms (17.3%-17.5% each), with Fever, Loss of smell, and Cough marginally the most common — consistent with the synthetic dataset's design rather than reflecting any single dominant symptom.

7. Machine Learning Model

An end-to-end pipeline was built to connect to the MySQL outbreak database, prepare patient/case information, engineer clinically meaningful features, train classification models to predict high severity, evaluate the models, interpret the final Random Forest, and expose the prediction workflow through a Streamlit application.

7.1 Project Snapshot

Item	Detail
Database	MySQL database: outbreak_x_final
Main tables used	final_Patient, final_Medical_History, final_Case, final_Case_Symptom
Primary task	Binary classification: predict High_Severity (0/1)
Candidate models	Logistic Regression and Random Forest Classifier
Train/Test split	80% / 20%, stratified by target
Rows used for modeling	600,000 total; 480,000 train; 120,000 test
Encoded feature count	26 features after one-hot encoding
Target distribution	16.6% High Severity, 83.4% Not Severe
Best model for Severe detection	Random Forest
Deployment	Streamlit application + Joblib artifacts

7.2 Pipeline Overview

- **Data Extraction** — connected Python to MySQL via SQLAlchemy and loaded the four source tables into Pandas DataFrames.
- **Feature Engineering** — computed patient Age from BirthDate; mapped Symptom_Severity and Disease_Severity to ordinal scores (Mild=1, Moderate=2, Severe=3); aggregated per-case symptom counts and severities, and per-patient chronic-history summaries.
- **Data Integration** — merged case-level and patient-level information into one modeling table using left joins on Patient_ID and Case_ID, keeping every case even where supporting information was missing.
- **Missing-Value Handling** — computed Days_To_Diagnosis from Diagnosis_Date and Symptom_Onset_Date; filled missing numeric summaries with 0 and missing text fields with 'None' where that reflects an actual absence of data.
- **Target Engineering** — created the binary High_Severity target (1 = Severe/Critical, 0 = otherwise).
- **Encoding & Split** — applied one-hot encoding to categorical features (Gender, Occupation, Chronic_Disease, Previous_Conditions), then performed an 80/20 stratified train/test split.
- **Model Training** — trained Logistic Regression (baseline) and a Random Forest Classifier (200 trees, max depth 10, class_weight='balanced') to address the class imbalance.
- **Evaluation** — compared both models using classification report, confusion matrix, and ROC-AUC.
- **Interpretation** — analyzed built-in Random Forest feature importance and permutation importance to understand which features the model relied on.
- **Deployment** — saved the trained model and supporting metadata with Joblib, then built a Streamlit application that collects patient/clinical inputs and returns a severity prediction and probability.

7.3 Model Comparison & Results

Logistic Regression achieved 83% overall accuracy but predicted every test case as 'Not Severe', giving 0% recall for the Severe class — a clear example of accuracy being misleading under class imbalance. The Random Forest, helped by `class_weight='balanced'`, substantially improved detection of severe cases.

Model	Severe Precision	Severe Recall	Severe F1-score	ROC-AUC
Logistic Regression	0.00	0.00	0.00	0.684
Random Forest	0.31	0.49	0.38	reported alongside ROC curve

Random Forest Confusion Matrix

	Predicted: Not Severe	Predicted: Severe
Actual: Not Severe	77,931 (TN)	22,105 (FP)
Actual: Severe	10,246 (FN)	9,718 (TP)

7.4 Feature Importance

Both the built-in Random Forest feature importance and permutation importance consistently ranked Age as the dominant predictor (importance score 0.948; permutation importance 0.183), far ahead of the next-ranked features such as Days_To_Diagnosis, symptom severity, and chronic-condition severity. A descriptive check (mean High_Severity rate by age, and a Pearson correlation of ≈ 0.25 between Age and High_Severity) confirmed a genuine positive relationship — while noting that feature importance reflects predictive usefulness within the model, not medical causation.

7.5 Deployment

The trained Random Forest model, the valid categorical options, and numeric feature defaults were saved as Joblib artifacts (`random_forest_model.pkl`, `categorical_options.pkl`, `numeric_defaults.pkl`). A Streamlit application ('Outbreak Severity Prediction') loads these artifacts, collects patient and clinical inputs, reconstructs the exact one-hot encoded feature schema used in training, and returns a High Severity prediction with its probability.

7.6 Limitations & Recommendations

- Class imbalance: only $\sim 16.6\%$ of cases are High Severity; Random Forest recall for Severe (≈ 0.49) still misses a substantial share of true severe cases.
- Feature importance is not causality — Age should be described as the most influential predictor, not as a cause of severity.
- The application currently reconstructs training encodings manually; a production version should persist a complete preprocessing pipeline (e.g. a scikit-learn Pipeline) alongside the model.
- Date-sequence validation (diagnosis date not earlier than symptom onset) should be added to the application.
- This model is a project prototype for analytics/education and should not be used as a clinical diagnostic tool.

8. Pivot Tables & KPIs (Excel)

Pivot tables were built on the cleaned dataset (Excel data model with defined table relationships) to summarize recovery outcomes, treatment protocols, age patterns, and demographic/geographic distribution.

8.1 Average Recovery Days by Case Severity

Severity	Avg. Recovery Days
Critical	13.50
Mild	9.26
Moderate	13.20
Severe	19.27
Grand Total	12.26

8.2 Dose Totals by Treatment Protocol

Protocol	Total Expected Doses	Total Taken Doses
Protocol A	4,225,292	3,703,886
Protocol B	3,728,788	3,269,120
Protocol C	3,177,928	2,584,858
Protocol D	648,492	528,281
Protocol E	465,784	346,777
Supportive Protocol	2,861,784	2,510,456
Grand Total	15,108,068	12,943,378

8.3 Patients & Recovery by Age

The age breakdown (ages 6-89) shows recovery time gradually rising with age: younger age groups (under ~45) average around 11-11.5 recovery days, while older age groups (65+) average 13.5-14.6 days, with the oldest bands (80+) at the high end. Overall: 198,900 patients, 595,284 cases, average recovery 12.27 days.

Age Group	Total Patients	Total Cases	Avg. Recovery Days
Under 20	≈14,500	≈42,900	≈11.4
20-39	≈58,000	≈174,000	≈11.6
40-59	≈54,700	≈163,600	≈12.1
60-79	≈35,600	≈107,000	≈13.5
80+	≈36,100	≈107,800	≈14.5
Grand Total	198,900	595,284	12.27

8.4 Distribution by Location, Occupation & Gender

Cairo governorate cities (Shubra, Maadi, Nasr City, Heliopolis) and Giza cities carry the largest patient/case volumes among the governorates profiled. By gender, Male patients show a higher share of Moderate, Severe, and Critical cases than Female patients. By occupation, Office Worker (29,652), Student (27,783), and Retail Worker (23,857) are the largest patient groups, alongside a total of 198,503 patients across all listed occupations.

Gender	Moderate	Severe	Critical	Mild	Total Cases
Female	101,667	31,236	8,247	96,190	238,354
Male	153,702	46,741	12,685	144,088	358,683

9. Interactive Dashboard (Power BI)

A two-page Power BI dashboard was built on the cleaned data model to give an at-a-glance view of outbreak scale, severity, recovery, and geographic/demographic spread. KPI cards, slicers (Severity, Gender, Governorate, Occupation), and multiple chart types (column, line, pie, treemap, scatter) allow interactive exploration.

9.1 Page 1 — Overview

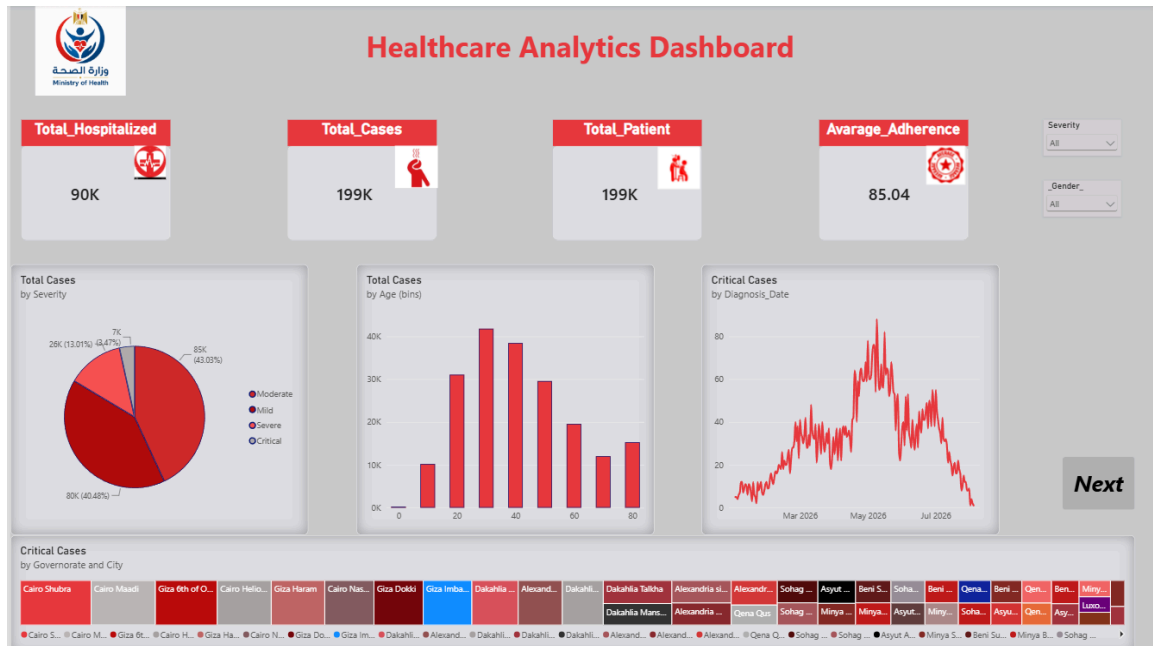


Figure 2 — Dashboard Page 1: Overview (Total Cases, Severity Split, Age Distribution, Critical Cases Trend)

- The dataset covers 199K patients and 199K cases in total, of which 90K (about 45%) were hospitalized.
- Average treatment adherence across all patients is 85.04%.
- By severity, cases split as Moderate 43.03%, Mild 40.48%, Severe 13.01%, and Critical 3.47% — meaning the large majority of cases (>83%) are Mild or Moderate.
- Case counts by age peak in the 20-40 age range, then decline in older age bins.
- Critical cases show a clear seasonal wave, rising sharply from around March 2026 and peaking near May 2026 before declining through July 2026 — consistent with an outbreak curve.
- Cairo, Giza, and Dakahlia governorates/cities carry the highest concentrations of critical cases in the treemap breakdown.

9.2 Page 2 — Severity, Recovery & Geography



Figure 3 — Dashboard Page 2: Severity Rate, Recovery by Protocol, Geographic & Gender Distribution

- 7K critical cases overall, an average recovery time of 12.27 days, and a Severe Rate of 12.96%.
- The Combination treatment protocol has the longest average recovery time (~14.5 days), followed by Single (~11.5 days) and Supportive protocols (~10 days) — suggesting Combination therapy is used for harder-to-treat cases rather than being less effective by itself.
- Total cases over time follow a single clear outbreak wave peaking around May 2026, matching the critical-cases trend on Page 1.
- Cairo and Giza governorates have markedly higher total case counts than Dakahlia, Alexandria, and Sohag.
- Male patients account for 60.14% of cases versus 39.86% for Female patients.
- The scatter plot of Crowding_Index against Total Cases shows no strong linear relationship — cases are scattered across a range of crowding levels rather than rising consistently with crowding, suggesting other factors (e.g. population size, healthcare access) also drive case counts.